



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

编译系统原理实验报告

编译系统原理实验一

刘家骥

年级：2022 级

专业：计算机科学与技术

指导教师：王刚

2024 年 9 月 23 日

摘要

本实验报告详细介绍了语言处理系统的完整工作过程，并探讨了编译流程的各个阶段，包括预处理器、编译器、汇编器和链接器的工作机制，探索了各个阶段的工作内容以及编译参数不同导致编译结果的差异。还尝试了编写 LLVM IR 程序以探讨 Sysy 语言的特性，编写了几个 Ssy 程序，并据此编写了相应的 ARM 汇编程序，来观察 Sysy 语言特性在汇编代码中的直接表现。

关键字：编译系统，LLVM IR，ARM 汇编

目录

一、 分工情况	1
二、 语言处理系统的完整工作过程	1
(一) 预处理器	1
1. 展开头文件	3
2. 加入大量注释和空格	3
3. 宏替换	3
4. 条件编译指令	4
5. 字符转义	5
(二) 编译器	5
1. 词法分析	5
2. 语法分析	6
3. 语义分析	7
4. 中间代码生成	7
5. 优化	9
6. 代码生成	11
(三) 汇编器	12
(四) 链接器	13
三、 LLVM IR 程序编写	14
(一) 中间语言编写与运行	14
1. 中间代码的编写	14
2. 中间代码的运行	14
(二) 数据类型	15
1. 二维数组的初始化	15
2. 二维数组的访问	15
3. 不同的数据类型	16
(三) 类型转换	16
(四) 函数	16
(五) 变量与常量的声明	16
(六) 分支与循环	17
四、 ARM 汇编程序	17
(一) 斐波那契数列	17
1. Sysy 编程	17
2. ARM 汇编编程	18
(二) 圆的周长	21
1. Sysy 编程	21
2. ARM 汇编编程	21

一、 分工情况

本次作业小题 1 各自负责，小题 2 LLVM IR 程序编写由王俊杰同学负责，小题 3 ARM 汇编程序的编写由刘家骥同学负责。

二、 语言处理系统的完整工作过程

以 C++ 编写的一个简单的阶乘程序为例，使用 clang 编译器进行编译，来探究编译过程程序代码如下

阶乘示例

```
1 #include<iostream>
2 using namespace std;
3 int main()
4 {
5     int i, n, f;
6     cin >> n;
7     i = 2;
8     f = 1;
9     while (i <= n)
10    {
11        f = f * i;
12        i = i + 1;
13    }
14    cout << f << endl;
15    return 0;
16 }
```

(一) 预处理器

预处理器对源代码进行文本级的替换和处理，以便编译器可以更有效地处理代码。处理源文件中的预处理指令（以 # 开头），包括 #define、#include、#if、#else、#endif 等等

在预处理阶段，程序的所有头文件将会被展开，宏将被替换，注释会被删除。

我们使用命令生成预处理后的文件

```
1 clang++ -E multiple.cpp -o multiple.i
```

查看 multiple.i 文件可以发现，只有 16 行的源代码扩展到了 30194 行。

```

30177     }
30178     # 2 "multiple.cpp" 2
30179     using namespace std;
30180     const int N = 100;
30181     int main()
30182     {
30183         int i, n, f;
30184         cin >> n;
30185         i = 2;
30186         f = 1;
30187         while (i <= n)
30188         {
30189             f = f * i;
30190             i = i + 1;
30191         }
30192         cout << f << endl;
30193         return 0;
30194     }
30195

```

图 1: 预处理

同时，我们还可以使用如下命令来生成预处理之后的文件

```
1 cpp main.c > main.i
```

这条命令使用独立的 C 预处理器工具 C Pre-Processor (调用时简写作 `cpp`), 与具体的编译器无关, 只进行源代码的预处理。

与 `clang++` 命令的区别在于, `clang+` 命令使用的是 Clang 编译器的预处理功能, `-E` 选项表示只执行预处理阶段, 而不是调用 C Pre-Processor

我们利用 `cpp` 命令生成一个 `multiple2.i` 的文件, 并使用 `diff` 命令查看 `multiple2.i` 和 `multiple.i` 两个文件有何不同

```

neo@tianxuanji:~/complierHomework/lab1$ diff multiple.i multiple2.i > diff.txt
neo@tianxuanji:~/complierHomework/lab1$

```

图 2: diff 命令

```

< # 1 "multiple.cpp" 2
< # 1 "/usr/bin/../lib/gcc/x86_64-linux-gnu/11/../../../../include/c++/11/iostream" 3
< # 37 "/usr/bin/../lib/gcc/x86_64-linux-gnu/11/../../../../include/c++/11/iostream" 3
---
> # 1 "/usr/include/c++/11/iostream" 3
> # 36 "/usr/include/c++/11/iostream" 3
>
> # 37 "/usr/include/c++/11/iostream" 3
>
> # 1 "/usr/include/x86_64-linux-gnu/c++/11/bits/c++config.h" 3
> # 278 "/usr/include/x86_64-linux-gnu/c++/11/bits/c++config.h" 3
11,12c15
< # 1 "/usr/bin/../lib/gcc/x86_64-linux-gnu/11/../../../../include/x86_64-linux-gnu/c++/11/bits/c++config.h" 3
< # 278 "/usr/bin/../lib/gcc/x86_64-linux-gnu/11/../../../../include/x86_64-linux-gnu/c++/11/bits/c++config.h" 3

```

图 3: 两种预处理的区别

可以看出，二者的预处理的头文件路径不相同

1. 展开头文件

观察经预处理后的代码可得，唯一的头文件 `iostream` 展开的具体路径如图

```
# 1 "multiple.cpp" 2
# 1 "/usr/bin/../lib/gcc/x86_64-linux-gnu/11/../../../../include/c++/11/iostream" 1
# 37 "/usr/bin/../lib/gcc/x86_64-linux-gnu/11/../../../../include/c++/11/iostream" 3
```

图 4: iostream 路径

我们尝试引入一个新的头文件 `cmath`，再次生成预处理之后的文件，可以看出，预处理后也将 `cmath` 展开

```
pow(long double __x, long double __y)
{ return __builtin_powl(__x, __y); }
# 412 "/usr/bin/../lib/gcc/x86_64-linux-gnu/11/../../../../include/c++/11/cmath" 3
```

图 5: cmath 路径

2. 加入大量注释和空格

尝试在源程序中加入大量的注释和空格，观察预处理过程会怎么处理这些字符。

如图6所示, 加入了一些注释和空格, 再经过预处理变为图7, 显然注释和空格均被消除

[illegible]

图 6: 加注释的源代码

```
30178 # 2 "multiple.cpp" 2
30179 using namespace std;
30180 int main()
30181 {
30182     int i, n, f;
30183     cin >> n;
30184     i = 2;
30185     f = 1;
30186
30187
30188     while (i <= n)
30189     {
30190         f = f * i;
30191         i = i + 1;
30192     }
30193     cout << f << endl;
30194
30195
30196
30197
30198
30199
30200
30201     return 0;
30202
30203 }
```

图 7: 加注释后预处理结果

3. 宏替换

编译器的预处理器在读取源代码时，会将所有符合 `#define` 宏定义的部分替换为相应的内容。它不会检查宏的类型或语法，只是简单地进行文本的替换。

```
#define GREETING "Hello, World!"
#define SQUARE(x) ((x) * (x))

int main() {
    int num1 = 5;
    std::cout << GREETING << std::endl;
    std::cout << "The square of " << num1 << " is: " << SQUARE(num1) << std::endl;

    return 0;
}
```

图 8: 加入宏定义代码

```
# 1 "define.cpp"
# 1 "<built-in>" 1
# 1 "<built-in>" 3
# 404 "<built-in>" 3
# 1 "<command line>" 1
# 1 "<built-in>" 2
# 1 "define.cpp" 2

int main() {
    int num1 = 5;
    std::cout << "Hello, World!" << std::endl;
    std::cout << "The square of " << num1 << " is: " << ((num1) * (num1)) << std::endl;

    return 0;
}
```

图 9: 预处理结果

显然可以看出 GREETING 在预处理后被替换为了“Hello, world!”, 而 SQUARE 也变成了两个变量相乘的形式。

4. 条件编译指令

我们新建一个 cpp 文件来观察条件编译指令在预处理时的变化。

```
int main() {
    #ifdef DEBUG
        printf("Debug mode is enabled.\n");
    #else
        printf("Hello, world!\n");
    #endif
    return 0;
}
```

```
# 1 "define.cpp"
# 1 "<built-in>" 1
# 1 "<built-in>" 3
# 404 "<built-in>" 3
# 1 "<command line>" 1
# 1 "<built-in>" 2
# 1 "define.cpp" 2
int main() {
    printf("Hello, world!\n");
    return 0;
}
```

图 10: 加注释的源代码

图 11: 加注释后预处理结果

```

1  #define DEBUG
2  int main() {
3      #ifdef DEBUG
4          printf("Debug mode is enal
5      #else
6          printf("Hello, world!\n");
7      #endif
8      return 0;
9  }
10

```

图 12: 加注释的源代码

```

1  # 1 "define.cpp"
2  # 1 "<built-in>" 1
3  # 1 "<built-in>" 3
4  # 404 "<built-in>" 3
5  # 1 "<command line>" 1
6  # 1 "<built-in>" 2
7  # 1 "define.cpp" 2
8
9  int main() {
10
11      printf("Debug mode is enabled.\n");
12
13
14
15      return 0;
16  }
17

```

图 13: 加注释后预处理结果

显然可以看出，当没有宏定义 DEBUG 时，输出的时 helloworld，当有 DEBUG 的宏定义时，输出的是 Debug mode is enabled.

5. 字符转义

预处理器可能会将某些特殊字符（如转义字符等）替换为它们的实际字符表示形式，这可能会增加文件的大小

(二) 编译器

在编译阶段可以分为以下几部分

1. 词法分析: 词法分析器将源代码中的字符序列转化为词法单元 (tokens)，每个词法单元代表程序的最小语义单位
2. 语法分析: 将词法单元序列组织成语法树 (Syntax Tree)，检查代码是否符合编程语言的语法规则
3. 语义分析: 检查程序的语义是否正确，即变量、函数、类型的使用是否合理，程序的逻辑是否满足规则。会进行类型检查、作用域检查、函数和变量的声明与定义检查等
4. 中间代码生成: 将语法树转化为一种中间表示，这种表示通常比源代码更接近机器代码。常见的中间表示有三地址码或静态单赋值形式
5. 优化: 对中间代码进行各种优化，以提高代码的效率和性能

1. 词法分析

我们利用以下命令来输出词法单元，并将输出重定向到文件 tokens.txt。

```
1 clang++ -E -Xclang -dump-tokens multiple.cpp > tokens.txt 2>&1
```

-E 表示命令编译器只执行预处理阶段，而不进行编译、汇编或链接。> tokens.txt 将标准输出（即预处理生成的词法单元信息）重定向到文件 tokens.txt 中。

需要注意的是, `2>&1` 参数是指将标准错误输出(即错误或警告信息)也重定向到标准输出中, 如果没有这个参数的话, `tokens.txt` 中是输入不了信息的。

```

126698 1_brace '{' [StartOfLine] Loc=<multiple.cpp:4:1>
126699 int 'int' [StartOfLine] [LeadingSpace] Loc=<multiple.cpp:5:5>
126700 identifier 'i' [LeadingSpace] Loc=<multiple.cpp:5:9>
126701 comma ',' Loc=<multiple.cpp:5:10>
126702 identifier 'n' [LeadingSpace] Loc=<multiple.cpp:5:12>
126703 comma ',' Loc=<multiple.cpp:5:13>
126704 identifier 'f' [LeadingSpace] Loc=<multiple.cpp:5:15>
126705 semi ';' Loc=<multiple.cpp:5:16>
126706 identifier 'cin' [StartOfLine] [LeadingSpace] Loc=<multiple.cpp:6:5>
126707 greatergreater '>>' [LeadingSpace] Loc=<multiple.cpp:6:9>
126708 identifier 'n' [LeadingSpace] Loc=<multiple.cpp:6:12>
126709 semi ';' Loc=<multiple.cpp:6:13>
126710 identifier 'i' [StartOfLine] [LeadingSpace] Loc=<multiple.cpp:7:5>
126711 equal '=' [LeadingSpace] Loc=<multiple.cpp:7:7>
126712 numeric_constant '2' [LeadingSpace] Loc=<multiple.cpp:7:9>
126713 semi ';' Loc=<multiple.cpp:7:10>
126714 identifier 'f' [StartOfLine] [LeadingSpace] Loc=<multiple.cpp:8:5>
126715 equal '=' [LeadingSpace] Loc=<multiple.cpp:8:7>
126716 numeric_constant '1' [LeadingSpace] Loc=<multiple.cpp:8:9>
126717 semi ';' Loc=<multiple.cpp:8:10>
126718 while 'while' [StartOfLine] [LeadingSpace] Loc=<multiple.cpp:9:5>
126719 1 paren '(' [LeadingSpace] Loc=<multiple.cpp:9:11>

```

图 14: 词法分析

结果分析 “`int 'int' [StartOfLine] [LeadingSpace] Loc=multiple.cpp:5:5`”是指声明整数变量(int)的关键字。位于第 5 行, 第 5 列。

“`identifier 'i' [LeadingSpace] Loc=multiple.cpp:5:9`”是指标识符 `i`, 是一个整型变量。位于第 5 行, 第 9 列。剩下的变量 `n`, `f` 同理。

“`identifier 'cin' [StartOfLine] [LeadingSpace] Loc=multiple.cpp:6:5`”是指标识符 `cin`, 输入流对象, 用于从标准输入获取数据。位于第 6 行, 第 5 列。

“`while 'while' [StartOfLine] [LeadingSpace] Loc=multiple.cpp:9:5`”是指关键字 `while`, 表示 `while` 循环开始。位于第 9 行, 第 5 列。

2. 语法分析

通过此命令来获取, `-E` 参数表示只执行预处理阶段, `-Xclang` 表示直接传递参数给 Clang 编译器前端, 而 `-ast-dump` 表示生成并打印出程序的抽象语法树 (AST)

```
1 clang++ -E -Xclang -ast-dump multiple.cpp
```

```

-FunctionDecl 0x8b5748 <multiple.cpp:19:1, line:32:1> line:19:5 main 'int ()'
-CompoundStmt 0x8b60a0 <line:20:1, line:32:1>
| DeclStmt 0x8b5988 <line:21:5, col:16>
| | -VarDecl 0x8b5800 <col:5, col:9> col:9 used i 'int'
| | -VarDecl 0x8b5880 <col:5, col:12> col:12 used n 'int'
| | -VarDecl 0x8b5900 <col:5, col:15> col:15 used f 'int'
| -CallExpr 0x8b5bb0 <line:22:5, col:19> 'int'
| | -ImplicitCastExpr 0x8b5b98 <col:5> 'int (*)(const char *__restrict, ...)' <FunctionToPointerDecay>
| | | -DeclRefExpr 0x8b5b18 <col:5> 'int (const char *__restrict, ...)' lvalue Function 0x89dda8 'scanf' 'int (const char *__restrict, ...)'
| | | | -DeclRefExpr 0x8b5be0 <col:11> 'const char *' <ArrayToPointerDecay>
| | | | -StringLiteral 0x8b5a68 <col:11> 'const char[3]' lvalue "%d"
| | | -UnaryOperator 0x8b5ad8 <col:17, col:18> 'int *' prefix '&' cannot overflow
| | | -DeclRefExpr 0x8b5a88 <col:18> 'int' lvalue Var 0x8b5880 'n' 'int'
| -BinaryOperator 0x8b5c38 <line:23:5, col:9> 'int' lvalue '='
| | -DeclRefExpr 0x8b5bf8 <col:5> 'int' lvalue Var 0x8b5800 'i' 'int'
| | -IntegerLiteral 0x8b5c18 <col:9> 'int' 2
| -BinaryOperator 0x8b5c98 <line:24:5, col:9> 'int' lvalue '='
| | -DeclRefExpr 0x8b5c58 <col:5> 'int' lvalue Var 0x8b5900 'f' 'int'
| | -IntegerLiteral 0x8b5c78 <col:9> 'int' 1
-WhileStmt 0x8b5ef0 <line:25:5, line:29:5>
| -BinaryOperator 0x8b5d28 <line:25:12, col:17> 'bool' '<='
| | -ImplicitCastExpr 0x8b5cf8 <col:12> 'int' <LValueToRValue>
| | | -DeclRefExpr 0x8b5cb8 <col:12> 'int' lvalue Var 0x8b5800 'i' 'int'
| | | -ImplicitCastExpr 0x8b5d10 <col:17> 'int' <LValueToRValue>
| | | | -DeclRefExpr 0x8b5cd8 <col:17> 'int' lvalue Var 0x8b5880 'n' 'int'
-CompoundStmt 0x8b5ed0 <line:26:5, line:29:5>
| -BinaryOperator 0x8b5d48 <line:27:9, col:17> 'int' lvalue '='
| | -DeclRefExpr 0x8b5d18 <col:9> 'int' lvalue Var 0x8b5900 'f' 'int'
| | -BinaryOperator 0x8b5d68 <col:13, col:17> 'int' '*'
| | | -ImplicitCastExpr 0x8b5da8 <col:13> 'int' <LValueToRValue>
| | | | -DeclRefExpr 0x8b5d68 <col:13> 'int' lvalue Var 0x8b5900 'f' 'int'
| | | -ImplicitCastExpr 0x8b5dc0 <col:17> 'int' <LValueToRValue>
| | | | -DeclRefExpr 0x8b5d88 <col:17> 'int' lvalue Var 0x8b5800 'i' 'int'
| -BinaryOperator 0x8b5eb0 <line:28:9, col:17> 'int' lvalue '='
| | -DeclRefExpr 0x8b5e18 <col:9> 'int' lvalue Var 0x8b5800 'i' 'int'
| | -BinaryOperator 0x8b5e90 <col:13, col:17> 'int' '+'
| | | -ImplicitCastExpr 0x8b5e78 <col:13> 'int' <LValueToRValue>
| | | | -DeclRefExpr 0x8b5e38 <col:13> 'int' lvalue Var 0x8b5800 'i' 'int'
| | -IntegerLiteral 0x8b5e58 <col:17> 'int' 1
-CallExpr 0x8b6010 <line:30:5, col:21> 'int'
| -ImplicitCastExpr 0x8b5ff8 <col:5> 'int (*)(const char *__restrict, ...)' <FunctionToPointerDecay>

```

图 15: AST 示例

首先一开始的 FunctionDecl 是函数声明，表面这是 main 函数，而 0x8b5748 则是这个节点的内存地址，int() 表示 main 函数返回类型为 int，且不接受参数。

第二个红框中，DeclStmt 表示是声明语句，后面的 VarDecl 表示每个变量的具体声明此处为声明变量 i, n, f，且可以看出都为 int 型变量。

第三个红框中，WhileStmt 表示一个 while 循环，而 BinaryOperator 表明了运算符为 <=，参与比较的两个数是 i 和 n，即循环条件是 i <= n。再下方 CompoundStmt 表示 while 循环的主体，即循环内的代码块。

3. 语义分析

我们以语法分析产生的 AST 中的红框为例，DeclStmt 声明了三个 int 型的变量 i, n, f，而其中的 used 关键字表明这些变量在后续的程序中确实被使用，表明了变量的定义和引用的语义检查。

CallExpr 表示函数调用，而 DeclRefExpr 表示对函数 scanf 的引用。编译器检查了 scanf 的函数签名 'int (const char *__restrict, ...)' 确保函数签名匹配，之后确认接收的参数类型是 const char* 和变长参数，进行了参数类型检查

4. 中间代码生成

用如下命令生成中间代码文件

```
1 clang++ -S -emit-llvm multiple.cpp
```

```

1 ; ModuleID = 'multiple.cpp'
2 source_filename = "multiple.cpp"
3 target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
4 target triple = "x86_64-pc-linux-gnu"
5
6 %"class.std::ios_base::Init" = type { i8 }
7 %"class.std::basic_istream" = type { i32 (...)**, i64, %"class.std::basic_ios" }
8 %"class.std::basic_ios" = type { %"class.std::ios_base", %"class.std::basic_ostream"*, i8, i8, %"class.std::basic_streambuf"*, %"class.std::
9 %"class.std::ios_base" = type { i32 (...)**, i64, i64, i32, i32, i32, %"struct.std::ios_base::Callback_list"*, %"struct.std::ios_base::Wor
10 %"struct.std::ios_base::Callback_list" = type { %"struct.std::ios_base::callback_list"*, void (i32, %"class.std::ios_base"*, i32)*, i32, i
11 %"struct.std::ios_base::Words" = type { i8*, i64 }
12 %"class.std::locale" = type { %"class.std::locale::Impl"* }
13 %"class.std::locale::Impl" = type { i32, %"class.std::locale::facet"*, i64, %"class.std::locale::facet"*, i8** }
14 %"class.std::locale::facet" = type <{ i32 (...)**, i32, [4 x i8] }>
15 %"class.std::basic_ostream" = type { i32 (...)**, %"class.std::basic_ios" }
16 %"class.std::basic_streambuf" = type { i32 (...)**, i8*, i8*, i8*, i8*, i8*, %"class.std::locale" }
17 %"class.std::ctype" = type <{ %"class.std::locale::facet.base", [4 x i8], %struct.__locale_struct*, i8, [7 x i8], i32*, i32*, i16*, i8, [256
18 %"class.std::locale::facet.base" = type <{ i32 (...)**, i32 }>
19 %struct.__locale_struct = type { [13 x %struct.__locale_data*], i16*, i32*, i32*, [13 x i8*] }
20 %struct.__locale_data = type opaque
21 %"class.std::num_put" = type { %"class.std::locale::facet.base", [4 x i8] }
22 %"class.std::num_get" = type { %"class.std::locale::facet.base", [4 x i8] }
23
24 @_ZStL8_oinit = internal global %"class.std::ios_base::Init" zeroinitializer, align 1
25 @_dso_handle = external hidden global i8
26 @_ZSt3cin = external global %"class.std::basic_istream", align 8
27 @_ZSt4cout = external global %"class.std::basic_ostream", align 8
28 @llvm.global_ctors = appending global [1 x { i32, void (*), i8* }] [{ i32 65535, void (*) @_GLOBAL__sub_I_multiple.cpp, i8* @_ZStL8_oinit }]
29
30 ; Function Attrs: noinline uwtable
31 define internal void @__cxx_global_var_init() #0 section ".text.startup" {
32   call void @_ZNSt8ios_base4InitC1Ev(%"class.std::ios_base::Init"* noundef nonnull align 1 dereferenceable(1) @_ZStL8_oinit)
33   %1 = call i32 @__cxa_atexit(void (i8*)* bitcast (void (%"class.std::ios_base::Init"*)* @_ZNSt8ios_base4InitD1Ev to void (i8*)*), i8* getelementptr
34   ret void
35 }
36
37 declare void @_ZNSt8ios_base4InitC1Ev(%"class.std::ios_base::Init"* noundef nonnull align 1 dereferenceable(1)) unnamed_addr #1
38
39 ; Function Attrs: nounwind
40 declare void @_ZNSt8ios_base4InitD1Ev(%"class.std::ios_base::Init"* noundef nonnull align 1 dereferenceable(1)) unnamed_addr #2
41
42 ; Function Attrs: nounwind
43 declare i32 @__cxa_atexit(void (i8*)*, i8*, i8*) #3
44

```

图 16: 中间代码

这是整段代码全局变量的初始化部分，包括初始化全局 I/O 流对象，由于使用了 iostream 做头文件，所以与 stdio 相比此处代码较为繁杂。比如 std::ios_base::Init, std::basic_istream 等用于表示标准 C++ 库中的 I/O 流类型。

观察中间代码文件，首先是 main 函数的代码

```

46 define dso_local noundef i32 @main() #4 {
47   %1 = alloca i32, align 4
48   %2 = alloca i32, align 4
49   %3 = alloca i32, align 4
50   %4 = alloca i32, align 4
51   store i32 0, i32* %1, align 4
52   %5 = call noundef nonnull align 8 dereferenceable(16) @"class.std::basic_istream"* @_ZNSirsERi(@"class.std::basic_istream"* noundef nonnull align 8 dereferenceable(16))
53   store i32 2, i32* %2, align 4
54   store i32 1, i32* %4, align 4
55   br label %6
56
57 6:
58   %7 = load i32, i32* %2, align 4
59   %8 = load i32, i32* %3, align 4
60   %9 = icmp sle i32 %7, %8
61   br i1 %9, label %10, label %16
62
63 10:
64   %11 = load i32, i32* %4, align 4
65   %12 = load i32, i32* %2, align 4
66   %13 = mul nsw i32 %11, %12
67   store i32 %13, i32* %4, align 4
68   %14 = load i32, i32* %2, align 4
69   %15 = add nsw i32 %14, 1
70   store i32 %15, i32* %2, align 4
71   br label %6, !llvm.loop !6
72
73 16:
74   %17 = load i32, i32* %4, align 4
75   %18 = call noundef nonnull align 8 dereferenceable(8) @"class.std::basic_ostream"* @_ZNSolsEi(@"class.std::basic_ostream"* noundef nonnull align 8 dereferenceable(8))
76   %19 = call noundef nonnull align 8 dereferenceable(8) @"class.std::basic_ostream"* @_ZNSolsEPFRSoS_E(@"class.std::basic_ostream"* noundef nonnull align 8 dereferenceable(8))
77   ret i32 0
78 }
79
80 declare noundef nonnull align 8 dereferenceable(16) @"class.std::basic_istream"* @_ZNSirsERi(@"class.std::basic_istream"* noundef nonnull align 8 dereferenceable(16))

```

图 17: main 函数中间代码

alloca 是变量分配，其中%1, %2, %3, 分别是变量 i, n, f 的空间。

函数调用如 @_ZNSirsERi 和 @_ZNSolsEi 分别代表输入和输出的操作，@_ZNSirsERi 是标准库函数，用于从 std::cin 输入流中读取整数，存储到%2 中，也就是 n 中。

原代码 61 行处，br i1 %9, label %10, label %16, 由于%9 存储 icmp%7 和%8 的结果，所以循环判断条件是%7 <= %8，在条件为真时跳转到循环体 label %10。

5. 优化

代码优化是进行与机器无关的代码优化步骤改进中间代码，生成更好的目标代码。使用不同的 pass，可以实现不同的优化效果。

我们可以尝试-dce 死代码消除优化来执行指令，生成优化后的中间代码：

```
opt -S -dce multiple.bc -o multiple_dce.ll
```

然后把生成的文件和优化前进行比较，执行 diff 命令并将输出重定向进 txt 文件：

```
diff multiple.ll multiple_dce.ll > diff_dce.txt
```

导出的文件如下：

dce 后的差异

```

1 1c1
2 < ; ModuleID = 'multiple.c'
3 ---
4 > ; ModuleID = 'multiple.bc'

```

可以看到，除了模块 ID 命名上的差异，没有任何变化。因为源代码和中间代码比较简单，情况比较单一，客观上不存在死代码，所以执行死代码消除（Dead Code Elimination, DCE）的优化没有任何变化。

我们再次尝试-mem2reg 参数来对编译进行优化，它会将内存中的局部变量提升到寄存器中。使用指令

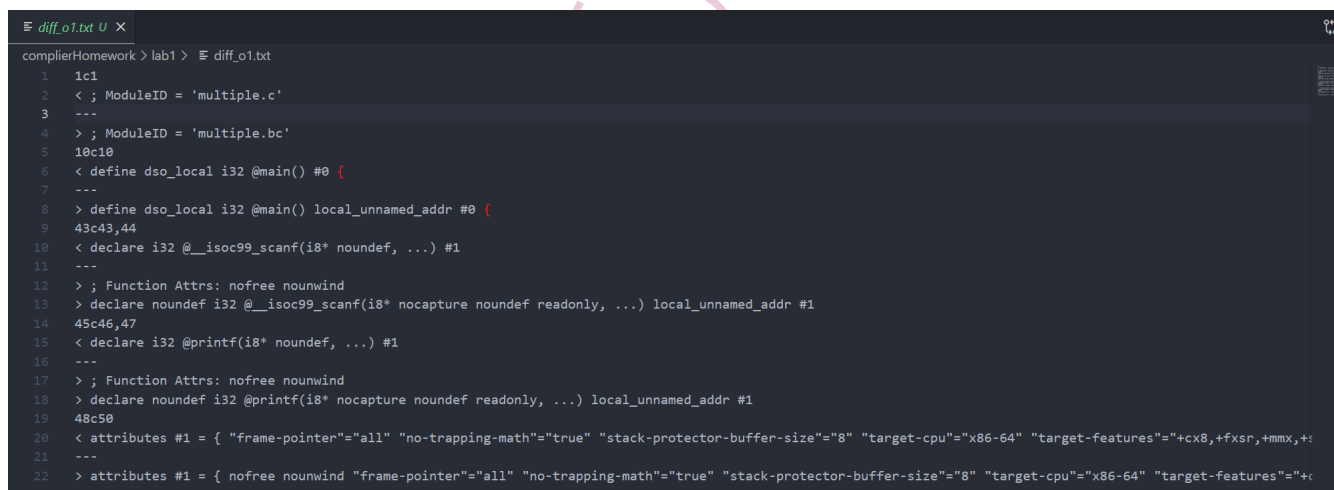
```
opt -S -mem2reg multiple.bc -o multiple_mem2reg.ll
diff multiple.ll multiple_dce.ll > diff_mem2reg.txt
```

得到的结果依然是除了模块 ID 命名上的差异，没有任何变化。

为了看到更明显的优化，我们使用 opt 工具进行-O1、-O2、-O3 三种级别的优化并将结果重定向进相应的 txt 文件，以便后续直接比较：

```
opt -O1 -S multiple.bc -o multiple_o1.ll
opt -O2 -S multiple.bc -o multiple_o2.ll
opt -O3 -S multiple.bc -o multiple_o3.ll
diff multiple.ll multiple_o1.ll > diff_o1.txt
diff multiple.ll multiple_o2.ll > diff_o2.txt
diff multiple.ll multiple_o3.ll > diff_o3.txt
```

查看 diff 指令输出重定向的文件后，可以直接观察到，三种优化策略生成的中间语言代码均相同，与原中间语言代码的差异均相同，差异如下图。



```
diff_o1.txt
1 1c1
2 < ; ModuleID = 'multiple.c'
3 ---
4 > ; ModuleID = 'multiple.bc'
5 10c10
6 < define dso_local i32 @main() #0 {
7 ---
8 > define dso_local i32 @main() local_unnamed_addr #0 {
9 43c43,44
10 < declare i32 @__isoc99_scanf(i8* noundef, ...) #1
11 ---
12 > ; Function Attrs: nofree nounwind
13 > declare noundef i32 @__isoc99_scanf(i8* nocapture noundef readonly, ...) local_unnamed_addr #1
14 45c46,47
15 < declare i32 @printf(i8* noundef, ...) #1
16 ---
17 > ; Function Attrs: nofree nounwind
18 > declare noundef i32 @printf(i8* nocapture noundef readonly, ...) local_unnamed_addr #1
19 48c50
20 < attributes #1 = { "frame-pointer"="all" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+s
21 ---
22 > attributes #1 = { nofree nounwind "frame-pointer"="all" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+s
```

图 18: 优化后代码差异

下面具体介绍其中的差异

首先是函数定义属性的差异

o1 后的差异

```
1 < define dso_local i32 @main() #0 {
2 ____
3 > define dso_local i32 @main() local_unnamed_addr #0 {
```

在优化后的 multiple_o1.ll 中，main 函数除了 dso_local 属性外，还多了 local_unnamed_addr 属性。local_unnamed_addr 表示该函数的地址不关心具体值，编译器可以更自由地优化地址使用。

对于 scanf 函数的声明也存在明显的差异

o1 后的差异

```

1 < declare i32 @__isoc99_scanf(i8* noundef, ...) #1
2
3 > ; Function Attrs: nofree nounwind
4 > declare noundef i32 @__isoc99_scanf(i8* nocapture noundef readonly, ...)
    local_unnamed_addr #1

```

观察可以得出，优化后的 scanf 函数声明中添加了 nofree、nounwind、nocapture、readonly 等属性。

- nofree：函数不会释放内存
- nounwind：函数不会引发异常
- nocapture：函数中的指针参数不会被外部保存
- readonly：表示传入的指针为只读

同样 printf 函数声明与 scanf 函数声明的变化一样，也是多出了这些参数。

这些额外声明的参数是编译器在优化时推断出的，帮助编译器更好地进行优化，有助于优化内存管理、异常处理等。减少了不必要的代码生成和内存使用，提高了程序的执行效率。

由于原代码过于简单，所以在进行优化后，各个中间语言代码之间的差异并不是很大。

6. 代码生成

利用以下命令，编译之前 c 代码生成的预处理文件 multiple.i，生成 arm 格式的目标代码

```

1 arm-linux-gnueabi-gcc multiple.i -S -o multiple.S

```

再使用如下命令生成 x86 架构的目标代码

```

1 gcc multiple.i -S -o multiplex86.S

```

由于 LLVM 面向多种架构的编译器基础设施，它可以生成适用于多种目标架构的代码。也可以使用如下命令生成目标代码

```

1 llc multiple.ll -o multiplellvm.S

```

并利用 diff 工具辅助分析三者的差别

```

1 diff multiple.S multiplex86.S
2 diff multiple.S multiplellvm.S
3 diff multiplellvm.S multiplex86.S

```

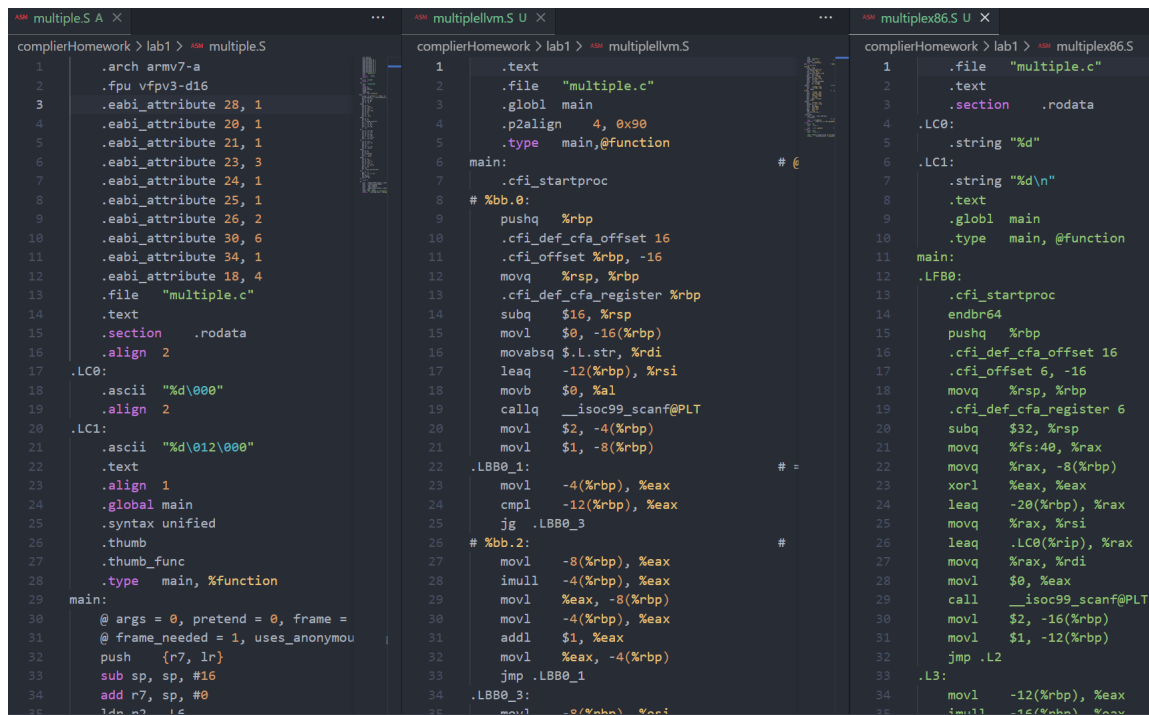


图 19: 三种汇编代码对比

图片左边的为 arm 架构的相应汇编代码。armv7-a 指定目标架构为 ARMv7-A，而后面的 vfpv3-d16 指定使用浮点处理器（FPU）的版本。

LC0、LC1 两个代码段，是用于 scanf 和 printf 函数的格式化字符串，在下面 main 代码段即为 main 函数主体内容，包括变量的初始化、赋值、运算以及循环等。

将 arm 架构汇编与 llvm 直接生成的目标代码对比，用 diff 命令以及观察区别发现，llc 编译器生成的 x86-64 汇编代码和 GCC 编译器生成的 ARM 汇编代码完全不同

llc 是 LLVM 编译器工具，它将 LLVMbitcode 文件编译成目标文件。llc 可以生成多种目标文件格式，包括针对不同体系结构的汇编代码，例如 x86、ARM 等。若不指定目标架构，llc 会生成与当前主机（你运行命令的机器）架构相对应的汇编代码，所以现在生成的是与我的电脑相对应的 x86-64 的汇编代码。

所以我们再将 gcc 编译器生成的 x86 汇编代码与 llc 编译器生成的汇编代码进行对比。

经过观察，二者用来实现乘法和循环的指令几乎相同，但是在一些细小的地方存在差异。llvm 生成的汇编代码将常量加载到寄存器中，而 GCC 使用常量池（LC0 和 LC1）

（三） 汇编器

汇编器将汇编代码翻译为目标机器指令

x86 格式汇编用以下命令

```
1 gcc multiplex86.S -c -o multiplex86.o
```

arm 格式的需要用到交叉编译

```
1 arm-linux-gnueabi-gcc multiple.S -o multiple.o
```

LLVM 可以直接使用 llc 命令同时编译和汇编 LLVM bitcode

```
1 llc multiple.bc -filetype=obj -o multiplellvm.o
```


(四) 链接器

链接器是在编译阶段之后，将多个目标文件（object files）和库文件链接在一起，生成最终的可执行文件。将编译器产生的各个独立的目标文件连接起来，确保程序中的函数调用、变量引用等正确地对应用到具体的内存地址。

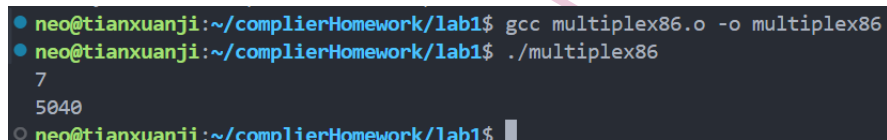
其中链接器可以分为静态链接器和动态链接器

- 静态链接器：将所有目标文件和依赖的静态库（如.a 文件）合并到一个完整的可执行文件中。在生成的可执行文件在运行时不依赖外部库，但是可执行文件体积较大，更新库文件时需要重新链接整个程序。
- 动态链接器：仅将目标文件链接在一起，外部库文件（如.so 文件）在程序执行时由操作系统加载。

在 x86 架构下，利用如下命令生成可执行文件

```
1 gcc multiplex86.o -o multiplex86
```

执行结果如下

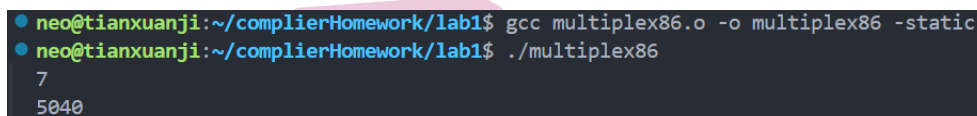


```
neo@tianxuanji:~/complierHomework/lab1$ gcc multiplex86.o -o multiplex86
neo@tianxuanji:~/complierHomework/lab1$ ./multiplex86
7
5040
neo@tianxuanji:~/complierHomework/lab1$
```

图 20: x86 可执行文件

-static 参数告诉编译器以静态链接方式链接程序，添加 -static 参数后将命令改写为

```
1 gcc multiplex86.o -o multiplex86 -static
```

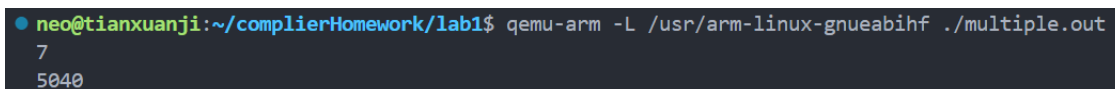


```
neo@tianxuanji:~/complierHomework/lab1$ gcc multiplex86.o -o multiplex86 -static
neo@tianxuanji:~/complierHomework/lab1$ ./multiplex86
7
5040
```

图 21: x86 可执行文件

在 qemu 模拟器中，利用如下命令生成 arm 架构下的可执行文件并执行

```
1 arm-linux-gnueabi-gcc multiple.S -o multiple.out
2 qemu-arm -L /usr/arm-linux-gnueabihf ./multiple.out
```



```
neo@tianxuanji:~/complierHomework/lab1$ arm-linux-gnueabi-gcc multiple.S -o multiple.out
neo@tianxuanji:~/complierHomework/lab1$ qemu-arm -L /usr/arm-linux-gnueabihf ./multiple.out
7
5040
```

图 22: arm 可执行文件

三、 LLVM IR 程序编写

(一) 中间语言编写与运行

LLVM IR 是由代码生成器自顶向下遍历逐步翻译语法树形成的，将任意语言的源代码编译成 LLVM IR，然后由 LLVM 后端对 LLVM IR 进行优化并编译为相应平台的二进制程序。我们通过自己来编写 LLVM IR 代码并运行来学习中间语言的特点。

1. 中间代码的编写

首先我们编写如下的代码：

```
1 target triple = "armv7-unknown-linux-gnueabi"
2 @.str = private unnamed_addr constant [14 x i8] c"Hello, SysY!\0A\00", align
  1
3 define dso_local i32 @main() #0 {
4   %1 = call i32 @__printf(i8*, ...) @printf(i8* noundef getelementptr inbounds ([14 x
      i8], [14 x i8]* @.str, i32 0, i32 0))
5   ret i32 0
6 }
7 declare i32 @printf(i8* noundef, ...) #1
```

在文件的头部指出目标的架构，然后定义常量字符串 @.str，之后定义主函数，使用 %1 来进行函数的调用，并返回 0。最后在主函数外面定义函数 printf。

2. 中间代码的运行

中间语言需要转成汇编语言 ARM，汇编语言需要汇编成目标文件，目标文件需要链接成可执行文件，可执行文件需要在 qemu 上执行，看到执行结果之后才能知道中间代码的编写是否正确。整个过程十分繁琐复杂，于是编写一个 run.sh 脚本：

```
1 #!/bin/bash
2 if [ "$#" -ne 1 ]; then
3     echo "Usage: $0 <file.ll>"
4     exit 1
5 fi
6 INPUT_FILE="$1"
7 OUTPUT_FILE="output"
8 EXECUTABLE="temp"
9 # 清理上次生成的文件
10 rm -f "$OUTPUT_FILE" "$EXECUTABLE"
11 # 生成目标文件
12 clang --target=arm-linux-gnueabi -mcpu=cortex-a8 -mfloat-abi=hard -o "
    $EXECUTABLE" "$INPUT_FILE" -lm
13 # 检查是否成功生成了可执行文件
14 if [ ! -f "$EXECUTABLE" ]; then
15     echo "Error: Failed to generate executable."
16     exit 1
17 fi
18 # 运行可执行文件
```

```

19 export LD_LIBRARY_PATH=/usr/arm-linux-gnueabi/lib
20 # 运行 QEMU
21 qemu-arm -L /usr/arm-linux-gnueabi/ ./"$EXECUTABLE"
22 # 检查 QEMU 运行是否成功
23 if [ $? -ne 0 ]; then
24     echo "Error: Failed to run executable with QEMU."
25     exit 1
26 fi
27 # 清理生成的文件
28 rm -f "$EXECUTABLE"

```

运行这个脚本，可以直接把中间语言的.ll 文件经过一系列步骤运行起来。

下面的代码因为篇幅过长，只讲解重要代码，代码的全文放在[这里](#)

(二) 数据类型

SysY 支持 int 和 float 两种基本类型：int 为 32 位有符号整数。float 为 32 位单精度浮点数。支持多维数组，数组元素可以是 int 或 float 类型，按行优先存储。根据这些特性，编写中间代码。

1. 二维数组的初始化

在中间语言里，采用在常量部分复制，在主函数申请空间，转成 i8，调用 memcpy 函数复制的步骤来实现数据分配到栈区。

数组的初始化

```

1 @__const.main.array = private unnamed_addr constant [2 x [3 x i32]] [[3 x i32
   ] [i32 1, i32 2, i32 3], [3 x i32] [i32 4, i32 5, i32 6]], align 4; 常量
   赋值
2 ; 主函数部分
3 %4 = alloca [2 x [3 x i32]], align 4; 申请空间
4 %5 = bitcast [2 x [3 x i32]]* %4 to i8* 转成 i8 便于函数操作
5 call void @llvm.memcpy.p0i8.p0i8.i32(i8* align 4 %5, i8* align 4 bitcast ([2
   x [3 x i32]]* @__const.main.array to i8*), i32 24, i1 false) ; 进行常量到
   栈区的复制操作

```

2. 二维数组的访问

对于二维数组的访问，采取先行后列的方式，采用 getelementptr，然后把地址加载到寄存器里。

数组的访问

```

1 %10 = getelementptr inbounds [2 x [3 x i32]], [2 x [3 x i32]]* %4, i32 0, i32
   1
2 %11 = getelementptr inbounds [3 x i32], [3 x i32]* %10, i32 0, i32 2
3 %12 = load i32, i32* %11, align 4

```

3. 不同的数据类型

采用 i32, float, double 等等的数据类型进行存储。

数组的访问

```
1 %6 = load i32, i32* %2, align 4
2 %7 = load float, float* %3, align 4
3 %8 = fpext float %7 to double
```

(三) 类型转换

实现隐式类型转换, 在不同类型的数据进行加和的时候, 会在加载的后面直接进行类型的转换, 包括 fpext 和 sitofp。

隐式转换

```
1 %5 = load i32, i32* %2, align 4
2 %6 = sitofp i32 %5 to float; 加载之后直接在另一个寄存器中转换
3 %7 = load float, float* %3, align 4
4 %8 = fadd float %6, %7
5 store float %8, float* %4, align 4
6 %9 = load float, float* %4, align 4
7 %10 = fpext float %9 to double
```

(四) 函数

可以实现函数的声明和调用, 实现函数的基本功能。

函数

```
1 define dso_local i32 @add(i32 noundef %0, i32 noundef %1) #0 {
2   %3 = alloca i32, align 4
3   %4 = alloca i32, align 4
4   store i32 %0, i32* %3, align 4
5   store i32 %1, i32* %4, align 4
6   %5 = load i32, i32* %3, align 4
7   %6 = load i32, i32* %4, align 4
8   %7 = add nsw i32 %5, %6
9   ret i32 %7
10 }
```

其中 define dso_local i32 @add(i32 noundef %0, i32 noundef %1) #0 这是定义一个局部可见的函数 add, 它返回一个 i32 类型 (32 位整数), 接受两个参数 %0 和 %1, 均为 i32 类型。把寄存器 0 和 1 都作为函数的参数使用。最后 ret i32 %7, 把寄存器的作为返回值返回。调用函数的时候, 采用 %4 = call i32 @add(i32 noundef 3, i32 noundef 4) 的调用方式, 传入参数并获得返回值, 放入寄存器当中。

(五) 变量与常量的声明

在主函数外面进行全局变量, 常量的定义, 并在主函数内部直接调用加载。

变量常量

```

1 @constVar = dso_local constant i32 100, align 4; constant 常量
2 @globalVar = dso_local global i32 50, align 4
3 define dso_local i32 @main() #0 {
4     %1 = load i32, i32* @constVar, align 4
5     %2 = load i32, i32* @globalVar, align 4
6     ; 其他代码
7 }

```

(六) 分支与循环

在控制代码各个步骤的过程中，采用代码块的方式，br 的跳转指令来实现分支和循环的功能。

分支

```

1     %3 = icmp sgt i32 %2, 0
2     br i1 %3, label %4, label %6
3 4:
4     ; 其他代码
5     br label %7
6 6:
7     br label %7

```

在%3 寄存器里面进行简单的 icmp sgt 判断后得到值，根据他来跳转到 4 或 6 的标签，实现分支。每个分支的最后都跳转到程序的下个地方，否则编译不通过。

循环

```

1     %8 = load i32, i32* %1, align 4
2     %9 = icmp sgt i32 %8, 0
3     br i1 %9, label %10, label %13
4
5 10:
6     ; 其他代码
7     %12 = add nsw i32 %8, -1
8     store i32 %12, i32* %1, align 4
9     br label %7
10 13:
11     ret i32 0

```

循环也是写成分支的形状，在循环体的最后写出循环变量的变化，然后再次分支看条件是不是可以到去前面。

四、 ARM 汇编程序

(一) 斐波那契数列

1. Sysy 编程

利用函数递归的方式实现斐波那契数列的计算。

输入 n 表示斐波那契数列的第几位，输出表示斐波那契数列的第几位

```
1 #include<stdio.h>
2 int Fib(int n)
3 {
4     if(n == 0)
5     {
6         return 0;
7     }
8     else if(n == 1)
9     {
10        return 1;
11    }
12    else
13    {
14        return Fib(n - 1) + Fib(n - 2);
15    }
16 }
17
18 int main()
19 {
20     int n;
21     printf("需要计算斐波那契数列的第几位:");
22     scanf("%d", &n);
23     if(n < 0)
24     {
25         printf("输入错误\n");
26         return 0;
27     }
28     printf("第%d项为:%d\n", n, Fib(n));
29     return 0;
30 }
```

2. ARM 汇编编程

编写等价的汇编代码如下

```
1 .arch armv7-a
2 .text
3
4 Fib:
5     push {lr}           @ 保存返回地址
6     cmp r0, #0
7     beq ifzero          @ 如果 r0 == 0, 返回 0
8     cmp r0, #1
9     beq ifone           @ 如果 r0 == 1, 返回 1
10
11     push {r0}           @ 保存当前 r0 的值
12     sub r0, r0, #1
```

```

13     bl Fib                @ 第一次递归调用, 计算 Fib(n-1)
14     mov r1, r0            @ 将 Fib(n-1) 的结果存入 r1
15     pop {r0}              @ 恢复原始的 r0
16
17     push {r1}              @ 保存 Fib(n-1) 的结果
18     sub r0, r0, #2
19     bl Fib                @ 第二次递归调用, 计算 Fib(n-2)
20     pop {r1}              @ 恢复 Fib(n-1) 的结果
21     add r0, r0, r1         @ Fib(n) = Fib(n-1) + Fib(n-2)
22
23     pop {lr}              @ 恢复返回地址
24     bx lr                 @ 直接跳转指令, 不会保存返回地址
25
26 ifzero:
27     mov r0, #0            @ 返回 0
28     pop {lr}
29     bx lr
30
31 ifone:
32     mov r0, #1            @ 返回 1
33     pop {lr}
34     bx lr
35
36
37 .globl main
38 main:
39     push {fp, lr}
40     sub sp, sp, #4        @ 为局部变量分配栈空间
41
42     ldr r0, =prompt
43     bl printf
44
45     ldr r0, =format
46     mov r1, sp            @ r1 存储栈顶的位置
47     bl scanf              @ 读取用户输入的整数值, 并将其存储在寄存器 r1 指向的
                           地址中
48
49     ldr r0, [sp]          @ r0 存储 sp 指向的值, 即用户输入的整数值
50     cmp r0, #0            @ if r0 <= 0
51     bge valid_input       @ 无符号大于或等于
52
53     ldr r0, =error_msg
54     bl printf
55     add sp, sp, #4
56     pop {fp, lr}
57     bx lr
58
59

```

```

60 valid_input:
61     bl Fib
62     mov r2, r0
63     ldr r0, =result
64     ldr r1, [sp]          @ 按照寄存器先后顺序输出,第一个格式化字符串是r1,第二个是r2
65     bl printf
66
67
68 end:
69     add sp, sp, #4
70     pop {fp, lr}
71     bx lr
72
73 .data
74 prompt:
75     .asciz "需要计算斐波那契数列的第几位:"
76 format:
77     .asciz "%d"
78 error_msg:
79     .asciz "输入错误"
80 result:
81     .asciz "第%d项为:%d\n"
82 debug:
83     .asciz "debug"

```

汇编代码带有 Sysy 语言的以下几个特性:

- 变量定义赋值: 利用 scanf 将输入数据 n 存储在栈中, 在代码中存储在了 sp 的位置。同时寄存器 r0 用于存储函数返回值或函数第一个参数。
- 基本算术运算: 使用了 add、sub 等基本的加减法指令, 用于计算地址偏移量、斐波那契数列和条件语句的边界检查
- 条件分支: 如通过 cmp 和 beq 指令实现相等条件下的跳转, 跳转到 ifzero 代码段, 实现 if 语句的功能
- 函数调用: 求斐波那契数列的过程进行了函数递归, 是一个函数不断调用自身的过程。以及在输入输出的时候调用 scanf 和 printf 函数
- 字符串: 在数据段通过 .asciz 伪指令声明的字符串, 用于打印输入输出以及格式化字符串

值得注意的是, 在对函数递归调用执行 Fib(n-1) 前, 需要 push r0 来保存此时 r0 的值, 在执行 Fib(n-2) 时将 r0 从栈中 pop 出来, 以保证数据的一致性。并且要注意正确保存和恢复寄存器状态。

撰写脚本编译运行可得

```

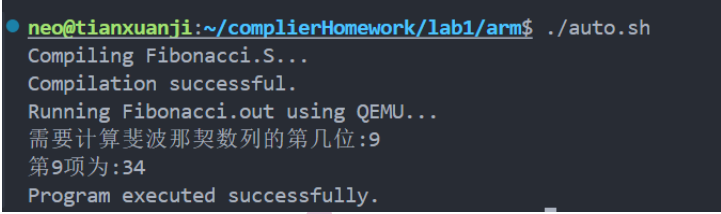
1 #!/bin/bash
2
3 # Step 1: Compile the ARM assembly file
4 echo "Compiling Fibonacci.S..."

```

```

5 arm-linux-gnueabi-gcc Fibonacci.S -o Fibonacci.out
6
7 # Check if compilation was successful
8 if [ $? -ne 0 ]; then
9     echo "Compilation failed."
10    exit 1
11 fi
12
13 echo "Compilation successful."
14
15 # Step 2: Run the compiled program using QEMU
16 echo "Running Fibonacci.out using QEMU..."
17 qemu-arm -L /usr/arm-linux-gnueabi-hf ./Fibonacci.out
18
19 echo "Program executed successfully."

```



```

neo@tianxuanji:~/complierHomework/lab1/arm$ ./auto.sh
Compiling Fibonacci.S...
Compilation successful.
Running Fibonacci.out using QEMU...
需要计算斐波那契数列的第几位:9
第9项为:34
Program executed successfully.

```

图 23: 斐波那契数列运行结果

(二) 圆的周长

1. Sysy 编程

定义了两个全局变量，用来表示圆周率 π 和圆的直径，主要涉及浮点数的运算
输入圆的直径 D ，输出圆的周长。

```

1 #include <stdio.h>
2 float pi = 3.14;
3 float D = 0.0; // 输入的直径
4
5 int main() {
6
7     scanf("%f", &D); // 输入D
8
9     float length = pi * D;
10    printf("周长为: %.2f\n", length);
11
12    return 0;
13 }

```

2. ARM 汇编编程


```
1  .arch armv7-a
2
3
4  .data
5  pi: .float 3.14
6  d: .float 0.0
7  length_format: .asciz "周长为: %.2f"
8  input_format: .asciz "%f"
9
10
11  .text
12  .global main
13
14  main:
15      push {fp, lr}
16
17      @输入D的值
18      ldr r0, =input_format
19      ldr r1, =d
20      bl scanf                @ 读取用户输入
21
22      @加载pi和d的值
23      ldr r0, =pi
24      ldr r1, =d
25      vldr.32 s0, [r0]        @ 将pi加载到s0
26      vldr.32 s1, [r1]        @ 将D加载到s1
27
28      @计算周长
29      vmul.f32 s2, s0, s1     @存储结果到s2
30
31      @输出
32      ldr r0, =length_format
33      vcvtf.f64.f32 d7, s2    @转为双精度浮点数,与printf兼容
34      vmov r2, r3, d7         @将双精度浮点数存储到r2和r3两个寄存器
35      bl printf
36
37      mov r0, #0
38      pop {fp, pc}
```

汇编代码带有 Sysy 语言的以下几个特性:

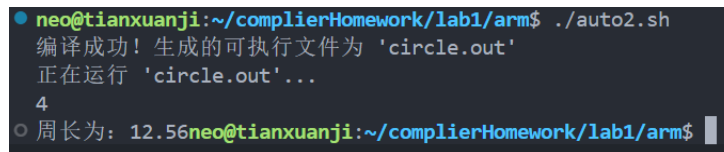
- 全局变量定义赋值: 在数据段定义了全局变量圆周率 pi 为浮点数 3.14 和要输入的浮点数圆的直径 d。并在 main 代码段中通过 ldr 指令将
- 基本浮点数运算: vmul.f32 单精度浮点数乘法指令, 来计算圆的周长, 即 $c = \pi * d$
- 条件分支: 如通过 cmp 和 beq 指令实现相等条件下的跳转, 跳转到 ifzero 代码段, 实现 if 语句的功能

- 浮点数指令：利用单精度浮点数装载指令 `vldr.32`，将常量 `pi` 和输入的直径 `d` 存入浮点数寄存器 `s0` 和 `s1` 中。
- 双精度输出：`printf` 只能输出双精度浮点数，因此需要将单精度浮点数转换为双精度，`vcvt.f64.f32` 将单精度浮点数转换为双精度，`s2` 被转换为双精度浮点数存储在 `d7` 中。

撰写脚本编译运行，脚本中应用了这两条命令

```
1 arm-linux-gnueabi-gcc -mfpv3 -o circle.out circle.S
2 qemu-arm -L /usr/arm-linux-gnueabi ./circle.out
```

运行结果如下图



```
neo@tianxuanji:~/complierHomework/lab1/arm$ ./auto2.sh
编译成功! 生成的可执行文件为 'circle.out'
正在运行 'circle.out'...
4
周长为: 12.56neo@tianxuanji:~/complierHomework/lab1/arm$
```

图 24: 求圆周长运行结果